

# Minimal Confusable Multisets: A Description of the Algorithms in `MinimalConfusableSets`

Lester (code), Claude (document reconstruction)

May 2026

## Abstract

This document gives a high-level description of the algorithms implemented in the `MinimalConfusableSets` subdirectory of the Echinocoder repository. The goal of that code is to search for, and determine the minimum size of, *confusable multisets*: pairs of equal-size multisets of vectors in  $\mathbb{R}^k$  that cannot be distinguished by a given family of encoding directions. The description here has been reconstructed from reading the source code rather than from the author's own notes; uncertain inferences are flagged with [?].

## Contents

|           |                                                                                |          |
|-----------|--------------------------------------------------------------------------------|----------|
| <b>1</b>  | <b>Background and motivation</b>                                               | <b>2</b> |
| <b>2</b>  | <b>The even-odd hypercube construction</b>                                     | <b>2</b> |
| <b>3</b>  | <b>Scaling the bad bats: unscaled and scaled representations</b>               | <b>3</b> |
| <b>4</b>  | <b>Vertex matches</b>                                                          | <b>3</b> |
| <b>5</b>  | <b>L-matrices</b>                                                              | <b>4</b> |
| <b>6</b>  | <b>The alpha-attacking matrix and collapse detection</b>                       | <b>4</b> |
| <b>7</b>  | <b>Computing the confusable pair</b>                                           | <b>5</b> |
| <b>8</b>  | <b>The search algorithm</b>                                                    | <b>5</b> |
| <b>9</b>  | <b>Pruning strategies</b>                                                      | <b>6</b> |
| 9.1       | Short-row pruning ( <code>prune_short_rows</code> ) . . . . .                  | 6        |
| 9.2       | Maximum-rows pruning ( <code>prune_max_rows</code> ) . . . . .                 | 6        |
| 9.3       | Pivot pruning ( <code>prune_pivots</code> ) . . . . .                          | 6        |
| 9.4       | Hash-based deduplication ( <code>remove_duplicates_via_hash</code> ) . . . . . | 6        |
| 9.5       | Equivalent-places symmetry reduction . . . . .                                 | 6        |
| <b>10</b> | <b>Vertex-match signature enumeration</b>                                      | <b>7</b> |

|                             |          |
|-----------------------------|----------|
| <b>11 Data flow summary</b> | <b>7</b> |
| <b>12 Notes and caveats</b> | <b>7</b> |

# 1 Background and motivation

[?] The broader Echinocoder project appears to be concerned with encoding multisets of vectors in  $k$ -dimensional space, and specifically with understanding when such an encoding fails to be injective. An encoding associates to each input multiset a set of dot products with a fixed collection of  $n$  *encoding direction vectors* (the “good bats”). The encoding is injective if no two distinct input multisets produce the same output.

The present sub-project addresses a specific obstruction to injectivity. If there exist  $M$  vectors  $\vec{q}_1, \dots, \vec{q}_M \in \mathbb{R}^k$  each of which is *orthogonal* to all the encoding direction vectors—these are the *bad bats*—then certain pairs of multisets built from the  $\vec{q}_i$  are invisible to the encoder. The code searches for the smallest such pairs, called *confusable multisets*.

The key parameters are:

- $k$ : the dimension of the ambient space,
- $M$ : the number of bad-bat vectors.

## 2 The even-odd hypercube construction

Given  $M$  vectors  $\vec{q}_1, \dots, \vec{q}_M \in \mathbb{R}^k$ , define the  $2^M$  *subset sums*

$$S_I = \sum_{i \in I} \vec{q}_i, \quad I \subseteq \{1, \dots, M\},$$

and partition them according to the parity of  $|I|$ :

$$\begin{aligned} E &= \{ S_I : |I| \text{ is even} \}, \\ O &= \{ S_I : |I| \text{ is odd} \}. \end{aligned}$$

Both  $E$  and  $O$  are multisets (a given point may appear with multiplicity  $> 1$  if different index sets  $I$  produce the same sum). By a simple symmetry argument  $|E| = |O| = 2^{M-1}$  counting multiplicity.

Let  $C$  be the “collision” multiset: for each point  $\vec{v}$ , its multiplicity in  $C$  is  $\min(\text{mult}_E(\vec{v}), \text{mult}_O(\vec{v}))$ . Define the *exclusive* parts

$$EE = E \setminus C, \quad OO = O \setminus C,$$

where subtraction is taken in the multiset sense. By construction  $|EE| = |OO|$ .

**Definition 1.** A *confusable multiset pair* is a pair  $(EE, OO)$  produced by the even-odd hypercube construction with  $|EE| = |OO| > 0$ .

**Remark 1.** The pair  $(EE, OO)$  is *confusable* in the sense that both multisets have the same multiset of dot products with any direction orthogonal to all  $\vec{q}_i$ . In particular, any encoding that uses only such orthogonal directions cannot distinguish  $EE$  from  $OO$ .

### Why $|EE|$ can be zero

If  $E$  and  $O$  coincide exactly as multisets (total cancellation), then  $C = E = O$  and  $EE = OO = \emptyset$ . The code calls this outcome *collapse*, and the corresponding configuration of bad bats is uninteresting. The search is for configurations that *avoid* collapse.

### 3 Scaling the bad bats: unscaled and scaled representations

In the code, the bad-bat vectors are separated into *directions*  $\vec{w}_1, \dots, \vec{w}_M \in \mathbb{R}^k$  (fixed integer vectors in “general position”) and *scalings*  $\alpha_1, \dots, \alpha_M \in \mathbb{R}$ , so that

$$\vec{q}_i = \alpha_i \vec{w}_i.$$

The *unscaled bat matrix*  $W$  is the  $M \times k$  matrix whose  $i$ -th row is  $\vec{w}_i$ . The *scaled bat matrix*  $B = \text{diag}(\alpha) W$  has  $i$ -th row  $\alpha_i \vec{w}_i$ .

The purpose of this split is to allow the problem of finding useful configurations to be formulated as a linear algebra question (Section 6).

The unscaled directions are chosen randomly in general position, meaning every subset of at most  $k$  of the  $\vec{w}_i$  is linearly independent. This genericity ensures that the null-space calculation (Section 6) captures the right constraints.

### 4 Vertex matches

**Definition 2.** A vertex match is a tuple  $\ell = (\ell_1, \dots, \ell_M) \in \{-1, 0, +1\}^M$  such that the number of  $+1$  entries is even and the number of  $-1$  entries is odd.

Geometrically, a vertex match encodes a coincidence between two subset sums: the  $+1$  positions form one summand and the  $-1$  positions form the other, so that the constraint  $\sum_i \ell_i \vec{q}_i = 0$  expresses the fact that those two subsets produce the same point. The parity condition (even  $+1$ s, odd  $-1$ s) ensures that one summand contributes to  $E$  and the other to  $O$ , producing the kind of coincidence needed for a non-trivial confusable pair. [?]

**Definition 3.** A vertex match  $\ell$  is useful (for given  $k$ ) if it has at least  $k + 1$  non-zero entries. The reason is that the sum of  $\leq k$  linearly independent vectors in  $k$ -dimensional space is always non-zero, so a match with only  $k$  or fewer active positions cannot hold for generic bad bats.

#### Tristate vs bistate vertex matches

Two variants of vertex match are present in the code:

- **Tristate:** entries in  $\{-1, 0, +1\}$  with the parity constraint above. This is the original and more general formulation.
- **Bistate:** entries in  $\{0, +1\}$  only (no  $-1$  entries). This is a more recent variant [?] corresponding to the `NO_MINUS_ONES_BRANCH` in git history. The geometric meaning of the bistate constraint is that a single subset sums to zero,  $\sum_{i \in I} \vec{q}_i = 0$ , rather than two subsets having equal sums.

At runtime exactly one variant is active, selected via `config.py`.

## Equivalent places

When the current partial L-matrix (see Section 5) has not yet broken any symmetry between some subset of the  $M$  bad-bat positions, those positions are *equivalent*: permuting them within the next row cannot produce anything inequivalent to what has already been generated. The code tracks these equivalence classes explicitly via the `EquivalentPlaces` class, and uses them to avoid redundant generation of permuted rows.

## 5 L-matrices

An *L-matrix* is a matrix  $L$  whose rows are vertex matches. Each row imposes one coincidence constraint; the full matrix imposes all of them simultaneously.

The ultimate output of the search is a “good” L-matrix: one for which (a) there exists a scaling  $\alpha$  consistent with all the constraints, and (b) the resulting confusable pair  $(EE, OO)$  is non-empty and as small as possible.

## 6 The alpha-attacking matrix and collapse detection

Substituting  $\vec{q}_i = \alpha_i \vec{w}_i$  into the  $r$ -th vertex-match constraint

$$\sum_{j=1}^M L_{rj} \vec{q}_j = \vec{0}$$

and writing out the  $k$  component equations gives, for each component  $\kappa$ :

$$\sum_{j=1}^M L_{rj} \alpha_j w_{j\kappa} = 0.$$

Collecting all  $R$  constraints and all  $k$  components into a single linear system  $A\alpha = 0$  defines the *alpha-attacking matrix*

$$A_{(r-1)k+\kappa, j} = L_{rj} \cdot w_{j\kappa}, \quad r = 1, \dots, R, \kappa = 1, \dots, k, j = 1, \dots, M.$$

This is computed by `alpha_attacking_matrix` in `confusable_multisets.py`.

The null space  $\mathcal{N}(A)$  is the set of valid scalings  $\alpha$ . Three cases arise:

1.  $\dim \mathcal{N}(A) = 0$ : the only solution is  $\alpha = 0$ ; all bad bats vanish. This is *complete collapse*; the matrix is discarded.
2.  $\dim \mathcal{N}(A) = M$ :  $A$  is the zero matrix, so  $\alpha$  is completely free. Any all-ones vector lies in the null space: no collapse.
3.  $0 < \dim \mathcal{N}(A) < M$ : check whether every coordinate direction  $e_j$  is non-zero in at least one basis vector of  $\mathcal{N}(A)$ . If so, a linear combination exists with all  $\alpha_j \neq 0$  (proved in the code comments, attributed to a OneNote entry). If some coordinate  $j$  is zero in every basis vector, then  $\alpha_j = 0$  is forced: bat  $j$  collapses.

## Finding a non-collapsing point in the null space

When the null space is non-trivial and non-collapsing, the code needs a specific point  $\alpha^* \in \mathcal{N}(A)$  with all  $\alpha_j^* \neq 0$ . This is done by `nonzero_lin_comb.combine_many`: starting from the first basis vector, it combines basis vectors one at a time, at each step choosing an integer multiplier  $\lambda$  (the smallest integer avoiding a finite set of “forbidden” values) that keeps all previously-established non-zero coordinates non-zero while activating the next coordinate.

## 7 Computing the confusable pair

Given the scaled bat matrix  $B = \text{diag}(\alpha^*)W$ , the code computes  $(EE, OO)$  via a *meet-in-the-middle* approach:

1. Split the  $M$  rows of  $B$  into a left half ( $M/2$  rows) and a right half.
2. For each half, enumerate all  $2^{M/2}$  subset sums, keeping track of the parity of the subset size. This gives four counters:  $L_{\text{even}}, L_{\text{odd}}, R_{\text{even}}, R_{\text{odd}}$ .
3. Convolve:

$$\begin{aligned} E &= (L_{\text{even}} * R_{\text{even}}) + (L_{\text{odd}} * R_{\text{odd}}), \\ O &= (L_{\text{even}} * R_{\text{odd}}) + (L_{\text{odd}} * R_{\text{even}}), \end{aligned}$$

where  $*$  denotes convolution of the counter-valued functions on  $\mathbb{R}^k$ .

4. Compute  $C, EE, OO$  as in Section 2.

This reduces the cost from  $O(2^M)$  to  $O(2^{M/2})$  at the expense of the convolution step.

## 8 The search algorithm

The outer loop, driven by `generate_viable_vertex_match_matrices` in `vertex_matches.py`, enumerates L-matrices via a depth-first search:

1. Start with the empty matrix.
2. At each level of the DFS, extend the current matrix by one row, chosen from all valid vertex matches respecting the current equivalence classes of bat positions.
3. Apply pruning tests (Section 9) to decide whether to yield the current matrix and/or recurse deeper.
4. For each yielded matrix, call the *decider function* to test for non-collapse and, if non-collapsing, to compute  $(EE, OO)$  and record  $|EE|$ .
5. Track the smallest  $|EE|$  found across all matrices and all choices of unscaled bat matrix.

## Robustness via multiple bat matrices

A collapse result for a specific unscaled bat matrix  $W$  does not rule out a non-collapsing solution for a different  $W$ . To guard against this, the demo driver (`tom_demo.py`) runs multiple independent `Rational_Decider` instances, each using a differently-seeded random bat matrix. A matrix is treated as collapsing only when all deciders agree. The comment in the code contains a probabilistic argument (incomplete at the time of writing) that this multi-decider approach can drive the false-collapse probability arbitrarily low.

## 9 Pruning strategies

Several pruning strategies are implemented to reduce the search space.

### 9.1 Short-row pruning (`prune_short_rows`)

If the reduced row echelon form (RREF) of the current matrix contains a row with between 1 and  $k$  non-zero entries, the matrix is discarded. The reasoning is that such a row would force a linear combination of  $\leq k$  specific bad-bat directions to vanish, which is impossible when those bats are in general position. This pruning is the most effective and is on by default.

### 9.2 Maximum-rows pruning (`prune_max_rows`)

The RREF of a viable L-matrix cannot have more than a certain number of rows, determined by  $M$  and  $k$ . The code can optionally stop deepening the DFS once this maximum is reached. This pruning is marked as “probably buggy/wrong” in the source.

### 9.3 Pivot pruning (`prune_pivots`)

Discards matrices whose RREF pivots are “too far to the right” given  $k$ . Also marked as “probably wrong” in the source.

### 9.4 Hash-based deduplication (`remove_duplicates_via_hash`)

The RREF of a matrix (optionally with columns scaled to a canonical form) is hashed. Matrices whose canonical RREF has been seen before are discarded. This is disabled by default with a warning that it can exhaust memory.

### 9.5 Equivalent-places symmetry reduction

As described in Section 4, the enumeration of rows respects the current equivalence classes of bat positions. This is not a pruning step per se but a reorganisation of the enumeration that avoids generating permutations of rows that are known to be equivalent.

A further (commented-out) idea in the source discusses pruning column-sign equivalences: if two L-matrices differ only in the signs of certain columns, and one precedes the other in the DFS, the later one need not be explored. This is noted as potentially very powerful but “frighteningly” hard to justify correctness for.

## 10 Vertex-match signature enumeration

Rather than enumerating vertex matches directly, the code first enumerates their *signatures*: the triple  $(e, o, z)$  giving the count of +1s, -1s and 0s respectively (with  $e + o + z = M$ ,  $e$  even,  $o$  odd in the tristate case). Signatures are generated in an order such that when converted to canonical tuples (sorted non-decreasingly) the result is in ascending lexicographic order.

For each signature, all distinct permutations of the corresponding canonical tuple are generated, subject to the current equivalent-places constraints. This two-level generation (signature, then permutations) is more efficient than iterating over all  $3^M$  possible tuples and filtering.

## 11 Data flow summary

The following diagram gives the overall data flow.

- Input**  $M$ ,  $k$ , and (optionally) specific unscaled bat matrices  $W$ .
- Step 1** Generate unscaled bat matrix  $W$  in general position (random integers, sampled from a Gaussian, with a given seed).
- Step 2** Enumerate L-matrices via DFS over vertex-match rows (`generate_viable_vertex_match_matrices`).
- Step 3** For each L-matrix, form the alpha-attacking matrix  $A = A(L, W)$  and compute  $\mathcal{N}(A)$ .
- Step 4** If non-collapsing, find  $\alpha^* \in \mathcal{N}(A)$  with all components non-zero (`nonzero_lin_comb`).
- Step 5** Compute  $B = \text{diag}(\alpha^*)W$  and evaluate  $(EE, OO)$  via meet-in-the-middle.
- Step 6** Record  $|EE|$  if it is the new minimum.
- Output** The minimum observed  $|EE|$ , together with the L-matrix, scalings, and scaled bat matrix that achieved it.

## 12 Notes and caveats

- All arithmetic is exact, using SymPy rational arithmetic throughout. This avoids floating-point errors in the null-space computation at the cost of speed.
- The search is *not* proved to be exhaustive (the pruning strategies marked “probably wrong” in the code may over-prune). The output is therefore a *upper bound* on the minimum confusable-set size, not necessarily the true minimum. [?]
- The `only_output_odd_ones` flag in `config.py` restricts the search to vertex matches with an odd number of +1 entries. The rationale for this restriction is not documented in the code, but may be related to a symmetry argument. [?]



- The `MinimalConfusableSets` subdirectory is not yet integrated with the rest of the Echinocoder project. The interface between it and “Tom’s” decider functions (referred to in the README and `deciders.py`) was under design at the time the code was last actively developed.